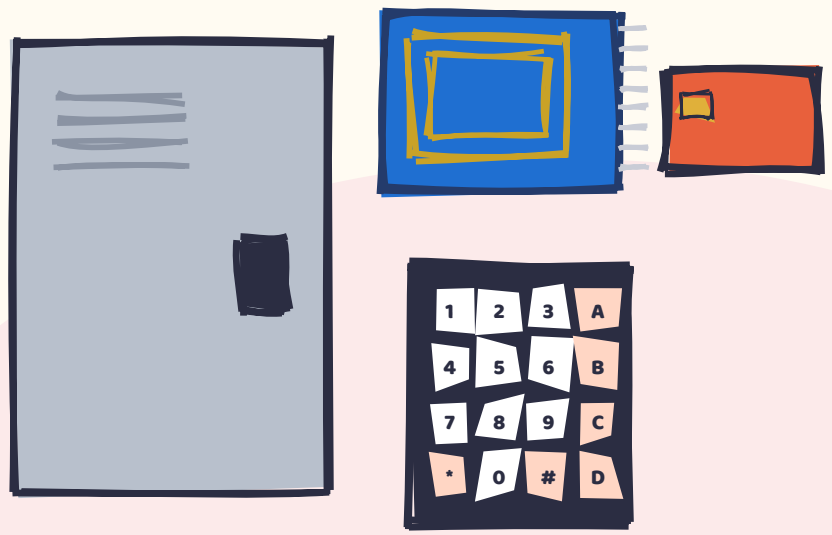
Tech Leaders · Class 8

Two-Factor Smart Locker

Card + PIN, a lockout, tamper alerts, then attack your own design

Authentication **State machine** **Threat model**



Did you know?
Most account break-ins use stolen or guessed passwords. A second factor blocks the large majority of those attacks.

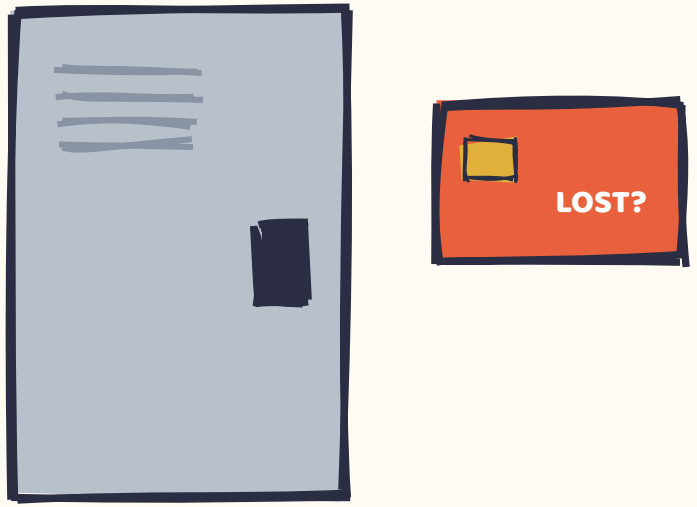
LEARN

The engineering problem

A lost card should not be a lost laptop

The school's problem

Students keep laptops and phones in lockers. Keys get copied and cards get lost. If **one** lost item opens the locker, it is not secure. And if someone forces it open, **nobody knows** until it is too late.



A bigger padlock?

A longer PIN?

Card AND PIN + alerts!

Today: you build a two-factor locker, then **switch sides** and try to break it like a security tester.

Learning objectives

By the end of this class I can…

1	**Authentication:** explain the 3 kinds of **authentication factor** and why two different ones beat one	**Proof:** sort 6 everyday logins into know / have / are
2	**Reading inputs:** read an **RFID card ID** over SPI and a **PIN** from a matrix keypad	**Proof:** Serial shows your card's UID and the keys you press
3	**Security logic:** build a **state machine** with time limits and a **lockout** after 3 wrong PINs	**Proof:** it refuses everything for 60 s after 3 wrong PINs
4	**Detect & log:** detect **tampering**, send an alert and keep a **timestamped log**	**Proof:** your phone gets an alert; the log shows every attempt
5	**Threat modelling:** think like an attacker: list **3 attacks**, rate them and **fix one**	**Proof:** a threat table plus one fix you built and tested

LEARN

Watch and think

Video time!



Two-Factor Authentication Explained in 3 Minutes

Common Craft · about 3 minutes · youtu.be/sPxAHV3P4IU

Watch for...

1. What are the two **factors**?
2. Why is a password alone not enough?
3. Give an example of 'something you have'.

Scan to watch

After the video: Our locker uses a keypad PIN and an RFID card. Which factor is which?

LEARN

Objective 1 · Authentication

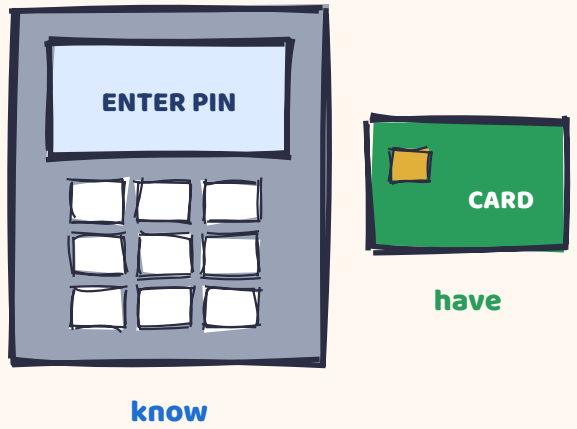
Three kinds of authentication factor

The idea

Something you KNOW: a PIN or password.
Something you HAVE: a card, a key, your phone.
Something you ARE: fingerprint, face.

Two-factor means two **different kinds**. A password + a security question is still one kind (both 'know'), so it is not two-factor.

Example: a bank ATM



To take cash you need the **card** (have) and the **PIN** (know). A thief who steals your wallet has the card but not the PIN. Someone who watched you type has the PIN but not the card.

LEARN

Objective 1 · Authentication

Why two different factors beat one strong one

The idea

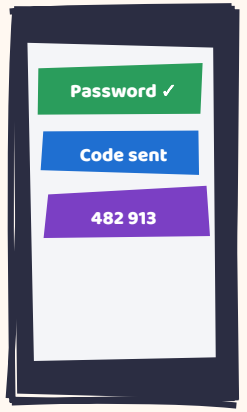
Each factor is stolen in a **different way**: passwords leak in data breaches and phishing; cards and phones are lost or stolen. An attacker rarely gets **both** at once.

A longer PIN protects against guessing, but not against someone who **watched you type it**.

Our design

Our locker: card = HAVE, PIN = KNOW. Two different kinds, so it is real two-factor.

Example: a leaked email password



stolen password

+ no phone

= no entry

Your password appears in a data breach. The attacker types it in, and the site sends a code to **your phone**. Without the phone, they are stuck. You get a warning and change the password.

Objective 2 · Reading inputs

RFID: the card answers with its ID

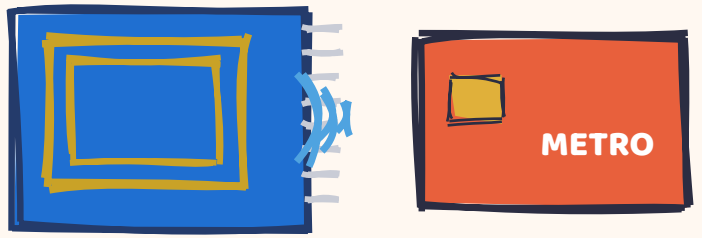
The idea

The **RC522 reader** sends out a radio field at 13.56 MHz. The card has no battery: the field **powers it**, and it replies with its **UID** (unique ID), e.g. **A3 4F 19 C2**.

The reader talks to the ESP32 over **SPI**, a fast 4-wire bus (SCK, MOSI, MISO, SS).

Warning
A UID is an ID number, not a secret. Cheap cards can be copied. Remember this for Objective 5.

Example: tapping a metro or bus card



reader powers the card by radio

You tap and the gate opens instantly, with no battery in the card. The reader powered the chip for a split second and read its number. Our locker does the same at a few centimetres.

Objective 2 · Reading inputs

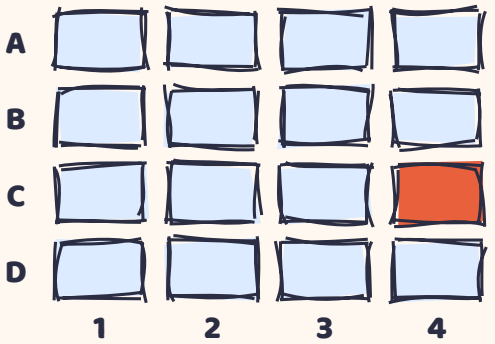
A matrix keypad: 16 keys, only 8 wires

The idea

Keys sit where **4 row wires** cross **4 column wires**. The ESP32 drives one column LOW at a time and checks which row goes LOW. Row + column = the key.

This **scanning** happens thousands of times per second. The **Keypad** library does it for us: **pad.getKey()**.

Example: a cinema seat



Row C, Seat 4

You do not need a number for each of 200 seats. 'Row C, Seat 4' finds it exactly. 16 keys need only 4 + 4 = 8 wires instead of 16 + 1.

Objective 3 · Security logic

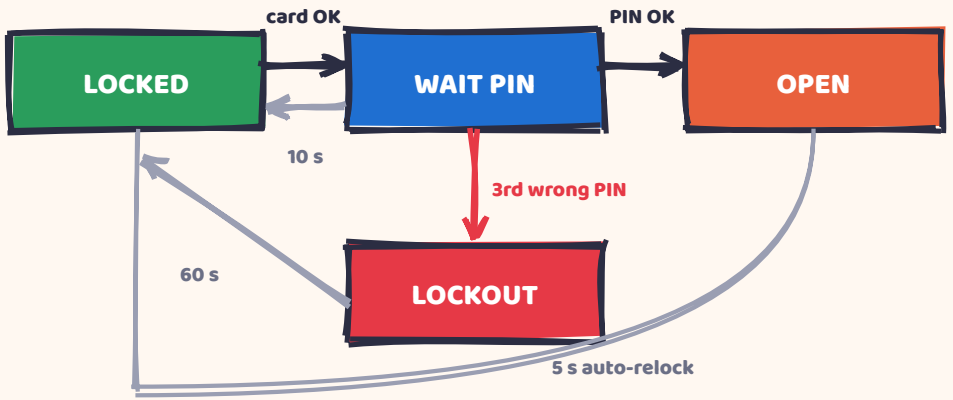
Security logic is a state machine

The idea

The locker is always in exactly one **state**. Inputs only matter if the current state allows them: a PIN typed while LOCKED (no card yet) is **ignored**.

Every state has a way out, including a **time limit**, so the locker can never get stuck open.

Example: our locker's states



Card OK moves LOCKED → WAIT PIN. PIN OK moves to OPEN, which relocks itself after 5 s. Three wrong PINs go to LOCKOUT for 60 s.

LEARN

Objective 3 · Security logic

Lockout turns guessing into waiting

The idea

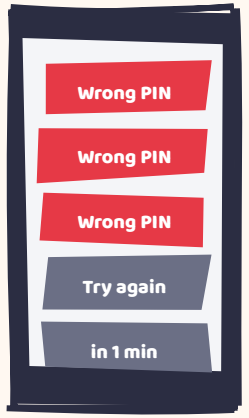
A 4-digit PIN has **10,000** possibilities. A fast attacker trying 1 per second would finish in under 3 hours.

With **3 tries, then 60 s lockout**: 3 tries per minute → 10,000 ÷ 3 ≈ 3,333 minutes ≈ **55 hours** worst case, and each lockout can trigger an **alert**.

Even better

Real systems double the lockout each time: 1, 2, 4, 8 minutes. Try that as a challenge.

Example: a phone lock screen



Type the wrong code too many times and the phone says **'Try again in 1 minute'**, then longer and longer. Guessing becomes so slow that it is not worth it.

LEARN

Objective 4 · Detect & log

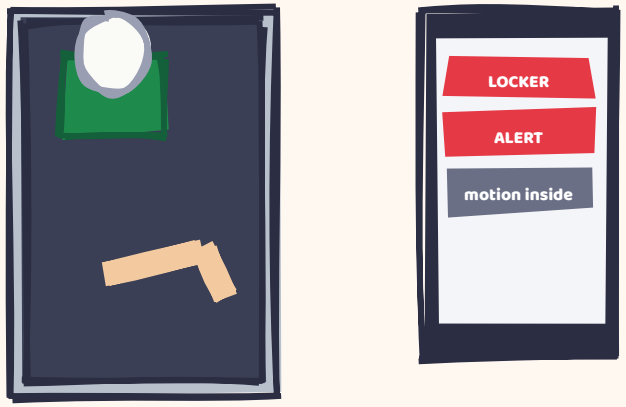
Tamper: motion where there should be none

The idea

Put the PIR sensor **inside** the locker. Movement inside is normal only in the **OPEN** state. Movement while LOCKED means the door was **forced**.

Then: buzzer + phone alert, but at most **one alert per 30 s**, or a fault would spam the owner until they ignore every alert.

Example: a shop's security tag



Paying removes the tag (authorised). Walking out with the tag still on sets off the gate alarm. The alarm depends on **state**: same movement, different meaning.

LEARN

Objective 4 · Detect & log

A log tells you what happened, and when

The idea

Every event gets a line with a **timestamp** (from internet time, NTP), what happened and the result:

```
[07/10 12:47:03] card 7B 01.. REFUSED
```

Log the **first part** of the UID only, and **never** log the PIN typed. A log must not become a new thing to steal.

Example: the visitor register at the school gate

08:02	A3 4F..	PIN ok
12:47	7B 01..	card no
12:48	7B 01..	card no
15:10	A3 4F..	PIN x3

Name, time in, time out, who they visited. Nobody checks it on a normal day, but after an incident it shows exactly **who was where, and when**.

LEARN

Objective 5 · Threat modelling

Threat modelling: think like the attacker

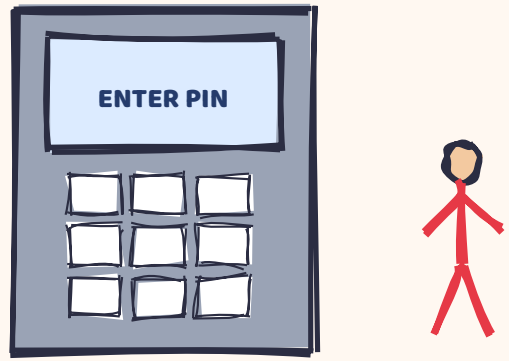
The idea

For each attack ask: **How hard** is it? **How bad** if it works? Fix the **easy + bad** ones first.

Our locker's weak spots:

· **Cloning**: copy the UID onto a blank card
· **Shoulder surfing**: watch the PIN
· **Brute force**: guess PINs
· **Power cut** or **Wi-Fi cut**: silence the alarm

Example: the hood over an ATM keypad



Banks studied how PINs were stolen and found people simply **watched**. The fix was cheap: a metal hood over the keypad. Small fixes for likely attacks beat clever fixes for rare ones.

Objective 5 · Threat modelling

When power fails: open or locked?

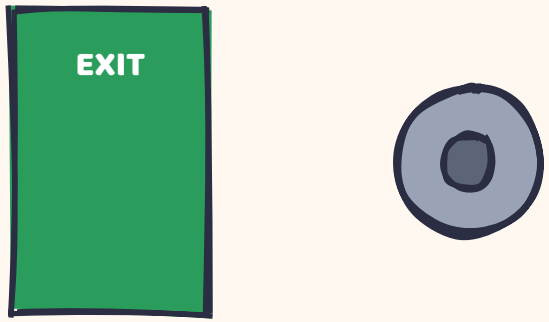
The idea

Fail-safe: unlocks when power fails, so **people** are safe.
Fail-secure: stays locked when power fails, so **things** are safe.

Our SG90 servo holds the latch only while powered. Pull the plug and the latch can be pushed. Is that what a locker should do?

Our answer
A locker holds things, not people, so it should fail-secure: a spring latch that stays shut, which the servo pulls open.

Example: fire exit vs bank vault



fail-safe

fail-secure

Fire exit doors with magnetic locks **release** in a power cut so nobody is trapped. A bank vault stays **shut**. Same question, opposite answers, because one protects people and the other protects money.

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Kit per group
2. Wire the card reader and screen
3. Wire keypad, latch and alarm
4. Read the card and set up the keypad
5. Log, alert and detect tampering
6. The security state machine
7. Switch sides: attack your own locker

Build it

Kit per group

ESP32
+ USB cable

RC522 + 2 cards
3.3 V module

4×4 keypad
8 wires

SG90 servo
latch + 5 V supply

PIR + buzzer
tamper alarm

OLED 0.96"
messages

Libraries: MFRC522 (miguelbalboa), Keypad (Mark Stanley), ESP32Servo, Adafruit SSD1306. **Phone:** ntfy app for alerts. **Locker:** a shoebox with a cardboard door.

Ethics rule
Only scan cards given to you for this class. Never read other people's ID or bank cards.

MAKE

Objective 2 · Build step 1 of 3

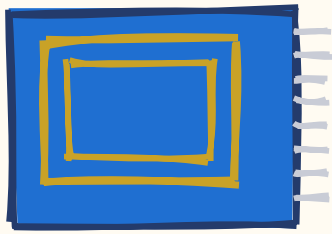
Wire the card reader and screen

a RC522 3.3V to **3V3** (never 5 V), GND to GND.

b RC522 SDA (SS) to **GPIO 5**, SCK to **18**, MOSI to **23**, MISO to **19**, RST to **17**. IRQ is not used.

c OLED SDA to **GPIO 21**, SCL to **22**, VCC to 3V3.

d Run the MFRC522 **DumpInfo** example, tap your card and copy its UID (e.g. A3 4F 19 C2).



RC522 RFID



Check: The UID appears every time you tap. The second card shows a different UID.

RC522 is a 3.3 V device. 5 V can damage it.

Objective 2 · Build step 2 of 3

Wire keypad, latch and alarm

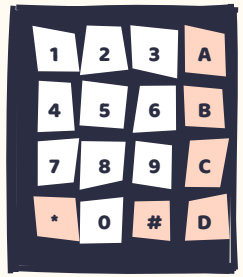
a Keypad pins 1 to 4 (rows) to **GPIO 13, 14, 27, 26**; pins 5 to 8 (columns) to **25, 33, 32, 15**.

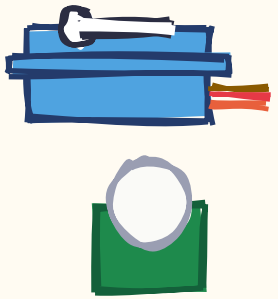
b Servo signal to **GPIO 4**. Power the servo from a **separate 5 V** supply; join the grounds.

c PIR OUT to **GPIO 34** (input-only pin: perfect), mounted **inside** the box.

d Buzzer + to **GPIO 16**, - to GND.

e Latch: the servo arm slides a lolly-stick bolt across the door. 0° = locked, 90° = open.





Check: Keypad example prints each key; servo moves 0° ↔ 90°.

Avoid GPIO 12 for the keypad: it is a boot pin and a pressed key at power-up can stop the ESP32 starting.

Objective 2 · Code part 1

Read the card and set up the keypad

```
#include <SPI.h>
#include <MFRC522.h>
#include <Keypad.h>
#include <ESP32Servo.h>
#include <Adafruit_SSD1306.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#define PIR 34
#define BUZ 16
MFRC522 rfid(5, 17);                          // SS, RST
char keys[4][4] = {{'1','2','3','A'}, {'4','5','6','B'},
                   {'7','8','9','C'}, {'*','0','#','D'}};
byte rowPins[4] = {13, 14, 27, 26}, colPins[4] = {25, 33, 32, 15};
Keypad pad = Keypad(makeKeymap(keys), rowPins, colPins, 4, 4);
Servo lock;
Adafruit_SSD1306 oled(128, 64, &Wire, -1);
const String OWNER_UID = "A3 4F 19 C2";      // from DumpInfo
const String OWNER_PIN = "4826";
enum State { LOCKED, WAIT_PIN, OPEN, LOCKOUT };
State state = LOCKED;
String typed;  int wrong = 0;  unsigned long t0 = 0;

String readCard() {                           // "" if no new card
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) return "";
  String uid;
  for (byte i = 0; i < rfid.uid.size; i++) {
    if (rfid.uid.uidByte[i] < 0x10) uid += "0";
    uid += String(rfid.uid.uidByte[i], HEX);
    if (i < rfid.uid.size - 1) uid += " ";
  }
  uid.toUpperCase();
  rfid.PICC_HaltA();
  return uid;
}
```

Card → text (O2)

Each UID byte becomes 2 hex characters, so it can be compared with **OWNER_UID**.

enum State (O3)

Names for each state make the logic readable: **state == WAIT_PIN**, not **state == 1**.

Honest note (O5)

A PIN in the code can be read by anyone with the board. Fix it later with a hash.

Objective 4 · Code part 2

Log, alert and detect tampering

```
void logEvent(String what) {                 // timestamped log line
  struct tm t;  char ts[20] = "no-time";
  if (getLocalTime(&t, 10)) strftime(ts, sizeof ts, "%d/%m %H:%M:%S", &t);
  Serial.printf("[%s] %s\n", ts, what.c_str());
}
void alert(String msg) {                     // push to the owner's phone
  logEvent("ALERT " + msg);
  if (WiFi.status() != WL_CONNECTED) return;
  WiFiClientSecure c;  c.setInsecure();     // classroom shortcut
  HTTPClient http;
  http.begin(c, "https://ntfy.sh/ptm-locker-9x2k");
  http.addHeader("Title", "Locker alert");
  http.POST(msg);
  http.end();
}
void beep(int n) {
  for (int i = 0; i < n; i++) { digitalWrite(BUZ, HIGH); delay(80);
                                digitalWrite(BUZ, LOW);  delay(80); }
}
void show(const char* msg) {
  oled.clearDisplay(); oled.setTextSize(2); oled.setTextColor(WHITE);
  oled.setCursor(0, 20); oled.println(msg); oled.display();
}
void checkTamper() {                         // motion INSIDE while locked
  static unsigned long last = 0;
  if (state == OPEN || !digitalRead(PIR)) return;
  if (millis() - last < 30000) return;       // max 1 alert per 30 s
  last = millis();
  beep(5);
  alert("Movement inside the locker while locked!");
}
```

State decides (O4)

Motion is only suspicious if **state != OPEN**. Same sensor reading, different meaning.

Rate limit

One alert per 30 s. An alarm that spams gets muted, and a muted alarm protects nobody.

Time from NTP

configTime() in setup gets real time from the internet, so the log is useful as evidence.

MAKE

Objective 3 · Code part 3

The security state machine

```
void go(State s, const char* msg) { state = s; t0 = millis(); typed = ""; show(msg); }

void setup() {
  Serial.begin(115200);
  SPI.begin();  rfid.PCD_Init();
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  lock.attach(4);  lock.write(0);            // start LOCKED
  pinMode(PIR, INPUT);  pinMode(BUZ, OUTPUT);
  WiFi.begin("SchoolLab", "********");
  configTime(5 * 3600, 0, "pool.ntp.org");   // Pakistan, UTC+5
  go(LOCKED, "Tap card");
}
void loop() {
  char k = pad.getKey();
  if (state == LOCKED) {
    String uid = readCard();
    if (uid != "") {
      logEvent("card " + uid.substring(0, 5) + ".. " + (uid == OWNER_UID ? "OK" : "REFUSED"));
      if (uid == OWNER_UID) go(WAIT_PIN, "PIN + #"); else beep(3);
    }
  } else if (state == WAIT_PIN) {
    if (millis() - t0 > 10000) go(LOCKED, "Tap card");    // factor 1 expires
    else if (k >= '0' && k <= '9') typed += k;
    else if (k == '#') {
      if (typed == OWNER_PIN) { wrong = 0; lock.write(90); logEvent("OPENED"); go(OPEN, "OPEN"); }
      else if (++wrong >= 3) { alert("3 wrong PINs, locked out"); go(LOCKOUT, "WAIT 60s"); }
      else { beep(2); typed = ""; logEvent("wrong PIN"); }
    }
  } else if (state == OPEN && millis() - t0 > 5000) {
    lock.write(0); go(LOCKED, "Tap card");               // auto relock
  } else if (state == LOCKOUT && millis() - t0 > 60000) {
    wrong = 0; go(LOCKED, "Tap card");
  }
  checkTamper();
}
```

Every state has an exit (O3)

WAIT_PIN expires after 10 s, OPEN relocks after 5 s, LOCKOUT ends after 60 s.

Log, but not secrets (O4)

We log 5 characters of the UID and never the PIN typed.

MAKE

Objective 5 · Build step 3 of 3

Switch sides: attack your own locker

Attack	How hard?	How bad?	Our fix
Card cloning	Easy with a cheap copier	High	
Shoulder surfing	Easy	High	
Brute force PIN	Slow (55 h)	High	
Pull the power	Easy	Door can be pushed	
Block Wi-Fi	Medium	Silent: no alerts	

Rules of the game
Attack only your own group's locker. Report every weakness you find. No damage.

Why this matters: real companies pay security testers to do exactly this before attackers do.

Your task: fill the last column, then **build one fix** and prove it works. Ideas: show **** on the OLED · alert when Wi-Fi drops · spring latch (fail-secure) · doubling lockout.

Test like an engineer

Troubleshooting

Symptom	Likely cause
No UID ever appears	RC522 on 5 V, or SS / RST pins swapped
ESP32 won't boot with keypad	a keypad wire on GPIO 12: move it
Keys come out wrong	row and column pins swapped in the arrays
ESP32 resets when servo moves	servo powered from the ESP32: use a separate 5 V supply
Tamper alert when you open it	PIR still 'hot' from the last opening: it holds HIGH a few seconds

Debug checklist

1. DumpInfo works?
2. Keypad example works?
3. Servo alone works?
4. Then combine

Build security systems **one part at a time**. When everything is combined, a bug in one part can look like a bug in another.

MAKE

Level up!

Apply what you learned

Choose your challenge



Second card: add a 'master' card for the teacher, logged separately (O1).



Doubling lockout: 60 s, then 120 s, then 240 s (O3).



Web log: serve the last 20 log lines at /log, protected by a password (O4).



Hashed PIN: store a SHA-256 hash, not the PIN itself, and compare hashes (O5).

Think like an engineer

Which is safer for a school locker: card + PIN, or fingerprint only? Think about lost cards, children's privacy and what happens when it fails.

Boss level

Use the card's secure memory sectors with a secret key instead of the UID, so a simple UID copy no longer works.

Objective check

Can I really do it? Prove it.

O1	Is 'password + secret question' two-factor? Is 'card + PIN'? Explain using know / have / are.	○ ○ ○
O2	How does a card with no battery send its UID? Why does a 4×4 keypad need only 8 wires?	○ ○ ○
O3	Draw the 4 states. What happens if you type the right PIN while in LOCKED (no card)?	○ ○ ○
O4	Why does the PIR alert only when the locker is not OPEN? Why is the PIN never logged?	○ ○ ○
O5	Name 3 attacks on your locker, rank them, and explain the fix you built. Fail-safe or fail-secure, and why?	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: O3: nothing. Keys are ignored in LOCKED; the card must come first. That is why order matters.

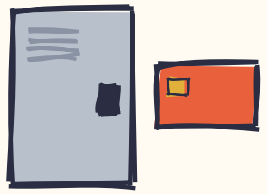
Show what you know

What did we learn?

- ☑ Two-factor = two **different kinds**: know + have (+ are)
- ☑ RFID cards are powered by the reader and reply with a **UID**; keypads are **scanned**
- ☑ A **state machine** with time limits and a **lockout** is the security logic
- ☑ Tamper = motion in the wrong **state**; logs need **time**, never secrets
- ☑ **Threat model** your own design; lockers should **fail-secure**

At home

Find 3 places where you use two-factor in daily life. Which factors are they?



? **Exit ticket:** Name one attack that two-factor does NOT stop, and how you would stop it.

Next class: TL-09 Sensor Data Logging & Analysis